

Instructions to create automata

April 15, 2026

Background definitions

The first goal is to write code to list all admissible ltt structures (these will give potential nodes of the automaton). The following definitions build up to defining an admissible ltt structure.

We are interested in a specific kind of colored graph that will have single-edge loops and multi-edges. We provide here this definition and then, for brevity, in the future, unless stated otherwise, we only mean this kind of graph:

Definition 0.1 (graph). By a *graph* we mean a graph in which each of (1)–(4) holds:

- 1) There is exactly one red vertex; all other vertices are purple.
- 2) There is exactly one red edge; all other edges are either black or purple. Edges colored red or purple are called *colored edges*.
- 3) Multi-edges are allowed, but for each pair of vertices there is at most one colored edge connecting that pair.
- 4) Single-edge loops are allowed, but only black edges can be loops.

In practice,

Definition 0.2 (notation). Define

$$\mathcal{L} = \{x_1, \bar{x}_1, \dots, x_5, \bar{x}_5\}, \quad \bar{\bar{x}}_j = x_j.$$

Assume

$$x_1 < \bar{x}_1 < \dots < x_5 < \bar{x}_5$$

is the increasing order on $\mathcal{L}$.

Definition 0.3 (distinguished admissible ltt structure). A *distinguished admissible ltt structure* is a connected graph G with:

- 5 pairs of vertices labeled precisely by the elements of $\mathcal{L}$ (duplicate labels are not allowed);
- x_1 is red and all remaining vertices are purple;
- x_1 is connected by a red edge to x_2 ;
- $\{\bar{x}_1, x_2, \bar{x}_2, \dots, x_5, \bar{x}_5\}$ is partitioned into three sets of size 3, and vertices in each partition element are pairwise connected by purple edges;
- each pair $\{x_j, \bar{x}_j\}$ is connected by a black edge (this black edge can be viewed as an oriented edge from $\{x_j$ to $\bar{x}_j\}$, and then $\bar{x}_j\}$ would notate this edge with a reverse orientation).

Definition 0.4 (epi-isomorphic, admissible ltt structure). Two colored graphs G_1, G_2 with vertices labeled by $\mathcal{L}$ are *epi-isomorphic* via bijection ϕ iff there is a bijection $\phi : \mathcal{L} \rightarrow \mathcal{L}$ such that:

- (i) for each $x \in \mathcal{L}$, the vertices labeled x and $\phi(x)$ have the same color;
- (ii) for each distinct pair of elements $x, y \in \mathcal{L}$, the vertices labeled $\phi(x), \phi(y)$ are connected by an edge of color C iff the vertices labeled x, y are connected by an edge of color C (black here is considered to be a color).

An *admissible ltt structure* is any graph epi-isomorphic to a distinguished admissible ltt structure.

Page 2

Place some (any) order on the admissible ltt structures. Let PLTT denote this ordered list.

Step 1. Start with a particular ltt structure G .

- $\text{GRAPHS} = \{G\}$;
- $\text{graphs}_1 = \{G\}$;
- $\text{boxes} = \{\text{graphs}_1\}$;
- layer 1 consists only of G , i.e. $\text{GRAPHS}_1 = \{G\}$.

[diagram in source image]

Remark. GRAPHS will ultimately be the list of all nodes in the automaton.

- each graphs_k will be a “box” of epi-isomorphic nodes;
- boxes will be the collection of these epi-isomorphism classes;
- before closing loops where nodes represent the same epi-isomorphism class of ltt structures, the automaton looks schematically like: GRAPHS_1 (layer 1), GRAPHS_2 (layer 2), GRAPHS_3 (layer 3), etc.

Page 3

Recursively (with respect to k), complete in PLTT order Steps 2–3 for each $G \in \text{GRAPHS}_k$.

Step 2. Find the 4 arrows (terminal ltt structures out of G). Let k denote the layer of G , i.e. $G \in \text{GRAPHS}_k$.

- Input the starting ltt structure G (an admissible ltt structure): vertices labeled by $\mathcal{L}$, with vertex colors and edge colors.
- Let d_r be the label of the red vertex.
- Let d_p be the label of the purple endpoint of the red edge.
- There are exactly two vertices connected to the d_p -vertex by purple edges:
 - 1) let d_1 be the label of the first;

2) let d_2 be the label of the second;

with order inherited from $\mathcal{L}$.

- For each of d_1, d_2 , there are two potential directed edges out of G (hence one potential terminal ltt structure for each such edge).

For $j \in \{1, 2\}$:

(2a) edge label: $d_r \rightarrow d_j \bar{d}_r$. The terminal ltt structure $G_{j;a}$ has:

- red vertex d_j and red edge $\{d_r, \bar{d}_j\}$;
- precisely these purple edges:
 - * a purple edge $\{d_j, d'\}$ whenever G has a colored edge $\{d_r, d'\}$;
 - * a purple edge $\{d, d'\}$ whenever G has a purple edge $\{d, d'\}$ and neither $d = d_r$ nor $d' = d_r$.

(2b) edge label: $d_j \rightarrow d_r \bar{d}_j$. The terminal ltt structure $G_{j;b}$ has:

- red vertex d_j and red edge $\{d_j, \bar{d}_r\}$;
- precisely these purple edges:
 - * a purple edge $\{d_r, d'\}$ whenever G has a colored edge $\{d_j, d'\}$;
 - * a purple edge $\{d, d'\}$ whenever G has a purple edge $\{d, d'\}$ and neither $d = d_r$ nor $d' = d_r$.

Let

$$\text{terminal}(G) := \{G_{1;a}, G_{1;b}, G_{2;a}, G_{2;b}\}$$

(repetitions allowed).

- Remove any graph from $\text{terminal}(G)$ that is not in PLTT.
- Each graph in $\text{terminal}(G)$ is in level $k + 1$.
- $\text{terminal}(G)$ and level $k + 1$ both inherit their order from PLTT.

Page 4

Step 3. In PLTT order, apply the following to each $G' \in \text{terminal}(G)$.

- If G' is epi-isomorphic to some $G'' \in \text{GRAPHS}$, but $G' \neq G''$:
 - $\text{GRAPHS} \leftarrow \text{GRAPHS} \cup \{G'\}$;
 - $\text{graphs}_j \leftarrow \text{graphs}_j \cup \{G'\}$ where $G'' \in \text{graphs}_j$;
 - add a “permutation arrow” $G' \rightarrow G''$ labeled by ϕ , where ϕ is the permutation of $\mathcal{L}$ from the epi-isomorphism.
- If there exists $G'' \in \text{GRAPHS}$ with $G' = G''$: add an arrow $G' \rightarrow G''$ with map $G' \mapsto G''$.
- For the remaining G' :
 - $\text{GRAPHS} \leftarrow \text{GRAPHS} \cup \{G'\}$;
 - $\text{graphs}_{i+1} = \{G'\}$ where $\text{graphs}_i \in \text{boxes}$ and $\text{graphs}_{i+1} \notin \text{boxes}$;
 - $\text{GRAPHS}_{k+1} \leftarrow \text{GRAPHS}_{k+1} \cup \{G'\}$.

Step 4. After Steps 2–3 are completed for each $G \in \text{GRAPHS}_k$, check whether $\text{GRAPHS}_{k+1} = \emptyset$.

If $\text{GRAPHS}_{k+1} = \emptyset$, terminate.

If $\text{GRAPHS}_{k+1} \neq \emptyset$, continue Steps 2–3 for each $G \in \text{GRAPHS}_{k+1}$.

Step 5. Output the automaton as nodes and edges (including ltt structure/map info). But what we really want is the strongly connected components.